

Python Programming with Artificial Intelligence

Machine Learning

Huangxun Chen @IoT&AI Thrust



ML Basics with Scikit-Learn

- Load Dataset, Split Data, Preprocess Data

Models in Scikit-Learn

- Supervised Learning Models, Unsupervised Learning Models

Model Evaluation & Selection

- Generalization: The Bias-Variance Trade-off
- Data-Splitting Protocol, Evaluation Metric, Cross-Validation

Hyperparameter Optimization

- Search Algorithm, Search Space



ML Basics with Scikit-Learn

What is Scikit-Learn?

Scikit-learn is an open-source Python library for machine learning. It has become the “de-facto” standard for general-purpose ML in Python.

Core Features

- Built on a solid foundation: Leverages NumPy, Pandas, SciPy, and matplotlib for performance and visualization.
- Comprehensive: Includes a wide range of algorithms for classification, regression, clustering, dimensionality reduction, and more.
- Well-documented: Excellent online documentation with many examples.
- Commercially usable: Open source with a permissive BSD license.

A Typical Workflow

Most machine learning projects in Scikit-learn follow these general steps.

- **Load Data:** Load your dataset, separating features ('X') from labels ('y').
- **Split Data:** Split the data into a training set and a test set.
- **Preprocess (Optional):** Create a preprocessing pipeline (e.g., scaling features).
- **Train Model:** Create an instance of a model and train it using the '.fit()' method on the training data.
- **Evaluate Model:** Use the trained model to make predictions with '.predict()' on the test set and assess its performance.

Loading Datasets

Scikit-learn comes with several toy datasets and helpers to fetch larger, real-world datasets.

```
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data # Features
y = iris.target # Labels

print(X.shape, y.shape) # (150, 4) (150,)
```

Loading a Toy Dataset (e.g., Iris)

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', version=1)
X, y = mnist["data"], mnist["target"]

print(X.shape, y.shape) # (70000, 784) (70000,)
```

Fetching a Real-World Dataset (e.g., MNIST)

Splitting Data

After loading the data, the next step is to **split it into a training set and a test set**. This allows us to train the model on one portion of the data and evaluate its **generalization performance** on another, unseen portion.

'train_test_split' function

```
from sklearn.model_selection import train_test_split

# Assume X and y are already loaded

# test_size=0.2 means 20% for testing, 80% for training
# random_state ensures the split is reproducible
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

Why Preprocess Data?

For example, features in a real-world dataset often have vastly different scales, ranges, and types

Problems with unscaled data:

- Models can be dominated by features with larger numeric ranges.
- Gradient-based algorithms (like neural networks) may converge slowly or get stuck.
- Distance-based algorithms (like KNN or K-Means) will produce biased results.

Example: House Price Prediction Dataset

- Age of the house: [0, 100] years
- Size of the house: [500, 5000] sqft
- Price of the house: [\$50,000, \$1,000,000]

The '`sklearn.preprocessing`' module provides a rich set of tools for common data transformation tasks.

- Scaling
- Normalization
- Encoding Categorical Features
- Discretization / Binarization
- Imputation of Missing Values
- Generating Polynomial Features
- Dimensionality Reduction (in '`sklearn.decomposition`')

Feature Scaling: StandardScaler

StandardScaler is one of the most common scalers.

It standardizes features by removing the mean and scaling them to unit variance. This results in a distribution with a mean of 0 and a standard deviation of 1

$$x' = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation of the feature column.

Important API Methods

- 'fit(X)': Computes the mean (μ) and standard deviation (σ) from the training data 'X'.
- 'transform(X)': Applies the learned scaling transformation to the data 'X'.
- 'fit_transform(X)': A convenient shortcut that performs both fitting and transforming in one step.

StandardScaler: Code Example

```
from sklearn import preprocessing
import numpy as np

X_train = np.array([[ 1., -1.,  2.],
                    [ 2.,  0.,  0.],
                    [ 0.,  1., -1.]])

# 1. Initialize the scaler
scaler = preprocessing.StandardScaler()

# 2. Learn the scaling parameters from the training data
scaler.fit(X_train)

# 3. Apply the transformation to the data
X_scaled = scaler.transform(X_train)

print("Scaled Data:\n", X_scaled)
# [[ 0.          -1.22474487  1.33630621]
#  [ 1.22474487  0.          -0.26726124]
#  [-1.22474487  1.22474487 -1.06904497]]

# You can also do it in one step
X_scaled_again = preprocessing.StandardScaler().fit_transform(X_train)
```

Other Common Scalers

Scikit-learn provides several other scalers for different use cases.

- 'MinMaxScaler': Scales each feature to a given range, typically [0, 1]
- 'RobustScaler': Uses statistics that are robust to outliers (median and interquartile range) to scale features. Use this if your data contains many outliers that might skew the mean and variance.

```
min_max_scaler = preprocessing.MinMaxScaler()
X_train_minmax = min_max_scaler.fit_transform(X_train)

# All values in X_train_minmax will be between 0 and 1
print(X_train_minmax)
```

Normalization

Normalization scales individual **samples (rows)** to have unit norm (length 1).

This is useful when the direction of the feature vector is more important than its magnitude.

```
X = [[ 1., -1.,  2.],
      [ 2.,  0.,  0.],
      [ 0.,  1., -1.]]
# fit() does nothing for Normalizer, but is needed for pipeline
# compatibility
normalizer = preprocessing.Normalizer().fit(X)
normalized_X = normalizer.transform(X)
print(normalized_X)
# [[ 0.40824829 -0.40824829  0.81649658]
#   [ 1.          0.          0.          ]
#   [ 0.          0.70710678 -0.70710678]]
```

Scaler vs. Normalizer

- **Scalers** operate on columns (features).
- **Normalizers** operate on rows (samples).



Encoding Categorical Features

Most machine learning models only accept numerical data. Categorical features (like 'male'/'female' or country names) must be converted to numbers.

OrdinalEncoder

- Encodes categorical features into an integer array.
- Best for ordinal data where order matters (e.g., low, medium, high).

```
from sklearn.preprocessing import OrdinalEncoder

# Data has a clear order: low < medium < high
X_ordinal = [['low'], ['medium'], ['high'], ['low']]
encoder = OrdinalEncoder(categories=[['low', 'medium', 'high']])
print(encoder.fit_transform(X_ordinal))
# [[0.], [1.], [2.], [0.]
```

For nominal features (where there's no order, like country names), **OneHotEncoder** is usually preferred to prevent the model from assuming a false ordering.

Discretization (KBinsDiscretizer)

Discretization (or binning) transforms continuous features into discrete intervals (bins). This can sometimes help models that struggle with continuous data.

```
from sklearn.preprocessing import KBinsDiscretizer

X = np.array([[ -3.,  5., 15],
               [  0.,  6., 14],
               [  6.,  3., 11]])

# n_bins: number of bins for each feature
# encode: 'ordinal' returns an integer for each bin
est = KBinsDiscretizer(n_bins=[3, 2, 2], encode='ordinal')
X_binned = est.fit_transform(X)

print(X_binned)
# [[0.  1.  1.]
#   [1.  1.  1.]
#   [2.  0.  0.]
```

■ Binarization (Binarizer)

Binarization is a special case of discretization where features are converted to either 0 or 1 based on a threshold.

```
X = [[ 1., -1.,  2.],
      [ 2.,  0.,  0.],
      [ 0.,  1., -1.]]

# Values > 0 become 1, others become 0
binarizer = preprocessing.Binarizer(threshold=0.0)
X_binarized = binarizer.fit_transform(X)

print(X_binarized)
# [[1., 0., 1.],
#   [1., 0., 0.],
#   [0., 1., 0.]]
```

Imputation: Handling Missing Values

Datasets often contain missing values (represented as 'NaN'). We must fill them in before feeding the data to a model. This is called imputation.

SimpleImputer

Replaces missing values using a simple **strategy** like the mean, median, or most frequent value of the respective column.

```
from sklearn.impute import SimpleImputer
import numpy as np

# Use 'mean' strategy to fill NaNs
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')

X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
imputer.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]]) # Fit on data
                                                    without NaNs in a column

# Transform the data with missing values
print(imputer.transform(X))
# [[ 7.    2.    3. ]
#   [ 4.    3.5   6. ]
#   [10.    3.5   9. ]]
```

Advanced Imputation

Simple strategies are fast, but more complex methods can often give better results by considering relationships between features.

KNNImputer

- Missing values are imputed using the mean value from the k-nearest neighbors found in the training set.
- It can be computationally expensive but very effective.

```
1 from sklearn.impute import KNNImputer
2 nan = np.nan
3 X = [[1, 2, nan], [3, 4, 3], [nan, 6, 5], [8, 8, 7]]
4 imputer = KNNImputer(n_neighbors=2)
5 print(imputer.fit_transform(X))
6 # [[1, 2, 4], [3, 4, 3], [5.5, 6, 5], [8, 8, 7]]
```

Generating Polynomial Features

- **PolynomialFeatures** can create a new feature matrix consisting of all polynomial combinations of the features with degree less than or equal to the specified degree.
- This is a powerful way to add non-linearity to models like Linear Regression.

```
from sklearn.preprocessing import PolynomialFeatures

X = np.arange(6).reshape(3, 2)
# X is [[0, 1], [2, 3], [4, 5]]

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# For features (X1, X2), this generates (1, X1, X2, X1^2, X1*X2, X2^2)
print(X_poly)

# [[ 1.  0.  1.  0.  0.  1.]
#  [ 1.  2.  3.  4.  6.  9.]
#  [ 1.  4.  5. 16. 20. 25.]]
```

Dimensionality Reduction with PCA

- Principal Component Analysis (PCA) is a widely used technique for dimensionality reduction.
- It identifies the axes (principal components) that account for the largest amount of variance in the data.

```
from sklearn.decomposition import PCA
import numpy as np
X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
pca = PCA(n_components=2)
pca.fit(X)

print(pca.explained_variance_ratio_)
# [0.99244289 0.00755711]
# The first component explains 99.2% of the variance!

print(pca.singular_values_)
# [6.30061232 0.54980396]
```

Use Case:

Speed up training by reducing the number of features.

Visualize high-dimensional data by projecting it down to 2 or 3 dimensions.

Mitigate the “curse of dimensionality”

In-Class Practice

Let's apply a full preprocessing pipeline to a subset of 300 passengers from the Titanic dataset.

- Load & Select: Load the Titanic dataset and randomly 300 samples.
Use features: 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', and 'Embarked'.
- Impute Missing Values: Fill missing values for all relevant features ('Age', 'Fare').
- Encode Categorical Features: Convert 'Sex' and 'Embarked' into numerical format.
- Scale All Features: Process all features so that their final values are in the range [0, 1].
- Take your fully preprocessed numerical feature matrix, use 'PCA(n_components=2)' to reduce its dimensionality to 2.
- Create a scatter plot of the two principal components.
- Color the points based on the 'Survived' column to see if there are any visible clusters of survivors vs. non-survivors



Models in Scikit-Learn

Supervised Learning Models

- Linear Regression
- Logistic Regression
- Support Vector Machines (SVM)
- Decision Trees
- Multi-layer Perceptron (MLP)

Unsupervised Learning Models (Clustering)

- K-Means
- Agglomerative Clustering
- DBSCAN
- Spectral Clustering
- Gaussian Mixtures

Linear Regression is a fundamental regression algorithm and a great starting point for supervised learning, used for predicting a continuous value.

- It's a “white box” model, meaning it's relatively easy to interpret.
- It assumes a linear relationship exists between the features and the target variable.
- The model learns by finding the best-fit line (or hyperplane) through the data.

The Equation: The model predicts an output $\hat{y}$ based on a linear combination of input features $x_1, \dots, x_n$ with learned weights (coefficients) $\theta_1, \dots, \theta_n$ and an intercept (bias) θ_0 .

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Linear Regression Example in scikit-learn

```
from sklearn.linear_model import LinearRegression

# Assume X_train, y_train are loaded
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# View the learned parameters
print("Intercept (theta_0):", lin_reg.intercept_)
print("Coefficients (theta_1...):", lin_reg.coef_)

# Make a prediction
some_data = X_test[0].reshape(1, -1)
prediction = lin_reg.predict(some_data)
print("Prediction:", prediction)
```

Why Not Use Linear Regression for Classification?

If we try to use a line to separate classes (e.g., “Benign” as 0, “Malignant” as 1):

- The model can predict values outside the 0-1 range (e.g., -0.5 or 1.5), which are meaningless as probabilities.
- The decision boundary (e.g., at 0.5) can be very sensitive to outliers.

Conclusion: We need a model designed for classification that outputs probabilities, such as Logistic Regression.

Logistic Regression

Logistic Regression is the baseline model for binary and multiclass classification problems.

It adapts linear regression by adding a “squashing” function to the output.

- 1 Calculate a score using a linear equation: $z = \theta^T x = \theta_0 + \theta_1 x_1 + \dots$
- 2 Pass this score through a **Sigmoid function** to transform it into a probability.

Sigmoid Function in Logistic Regression

The sigmoid function $\sigma(z)$ takes any real number and maps it to the range (0, 1), making it perfect for representing a probability.

Function and Properties

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- As $z \rightarrow \infty$, $\sigma(z) \rightarrow 1$.
- As $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$.
- If $z = 0$, $\sigma(z) = 0.5$. This is the decision boundary.

Take binary classification as an example

- The full model calculates the probability that a sample belongs to the positive class (class 1)

$$\hat{p} = \sigma(\theta^T x)$$

- Decision Rule: If $p \geq 0.5$, predict class 1. Otherwise, predict class 0

Logistic Regression Example in scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

# Load data and use two features for visualization
iris = load_iris()
X = iris.data[:, :2]
y = iris.target

# 'multinomial' enables Softmax Regression for multiclass problems
# 'C' is the inverse of regularization strength
softmax_reg = LogisticRegression(multi_class="multinomial",
                                solver="lbfgs", C=10)

softmax_reg.fit(X, y)

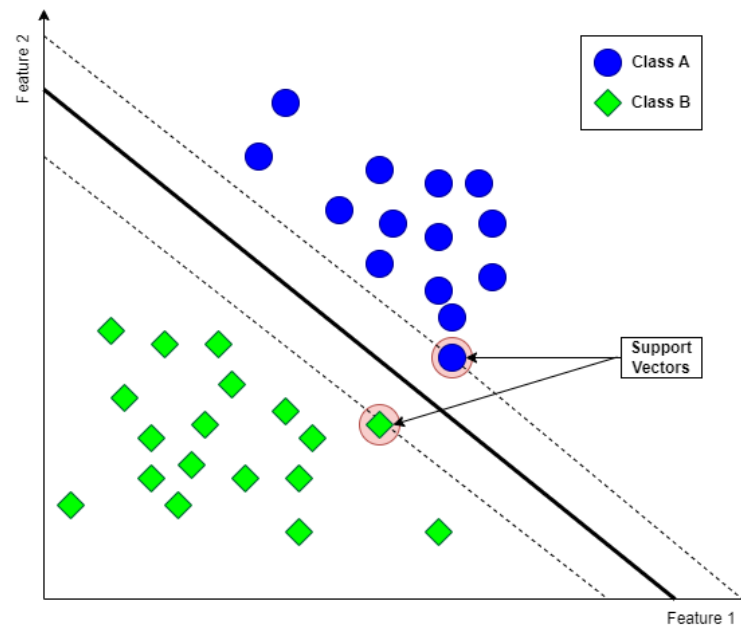
# Predict the class for a new flower
new_flower = [[5, 2]]
print("Predicted class:", softmax_reg.predict(new_flower))

# Predict the probabilities for each class
print("Probabilities:", softmax_reg.predict_proba(new_flower))
```

Support Vector Machines (SVM)

Support Vector Machine (SVM) is a powerful and versatile model, capable of performing linear or non-linear classification, regression, and even outlier detection, particularly well-suited for classification of complex, small- to medium-sized datasets.

The fundamental idea behind SVMs is to fit the widest possible “street” between the classes. The instances located on the edge of the street are called **support vectors**, and they entirely determine the decision boundary.



SVM Example in scikit-learn: The Kernel Trick

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.datasets import make_moons
# Generate non-linear moon-shaped data
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
# Create a pipeline to scale data then apply a polynomial SVM
poly_svm_clf = make_pipeline(
    StandardScaler(),
    SVC(kernel="poly", degree=3, coef0=1, C=5)
)
poly_svm_clf.fit(X, y)
```

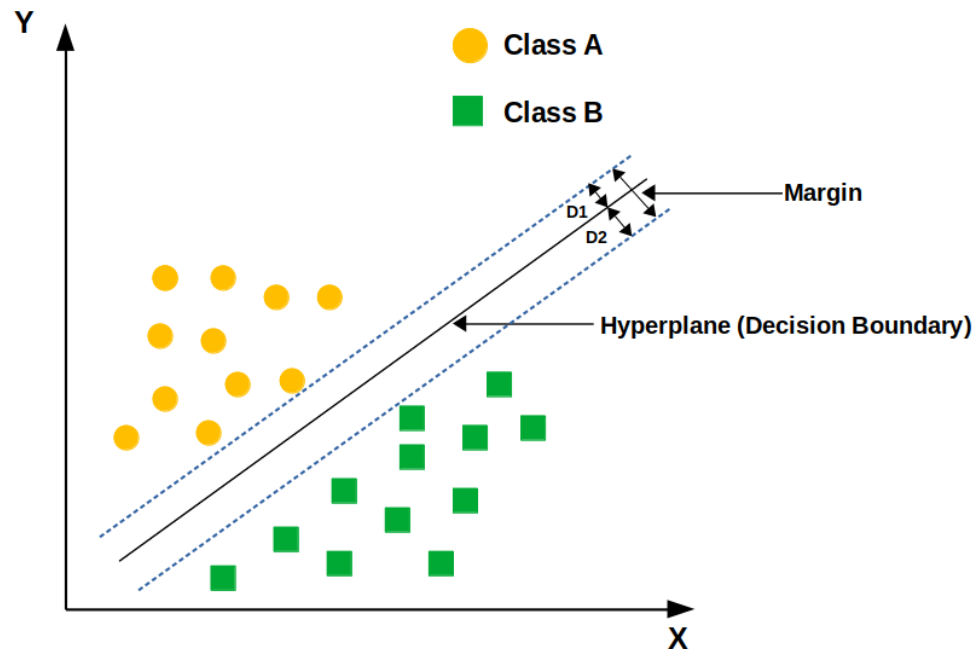
Polynomial Kernel

- For non-linear data, one approach is to add polynomial features. However, this can be computationally expensive.
- SVMs can use a mathematical technique called the **kernel trick** to achieve the same result without actually adding the features.
- We can also choose other Kernels, such as the Radial Basis Function (RBF) kernel.

The “C” Hyperparameter

In Scikit-learn’s SVM classes, the ‘C’ hyperparameter controls **Soft Margin Classification**.

- A **low C value** makes the margin wider but **allows for more margin violations**. This corresponds to **higher regularization**.
- A high C value makes the margin narrower but results in fewer margin violations. This corresponds to lower regularization and can lead to overfitting.



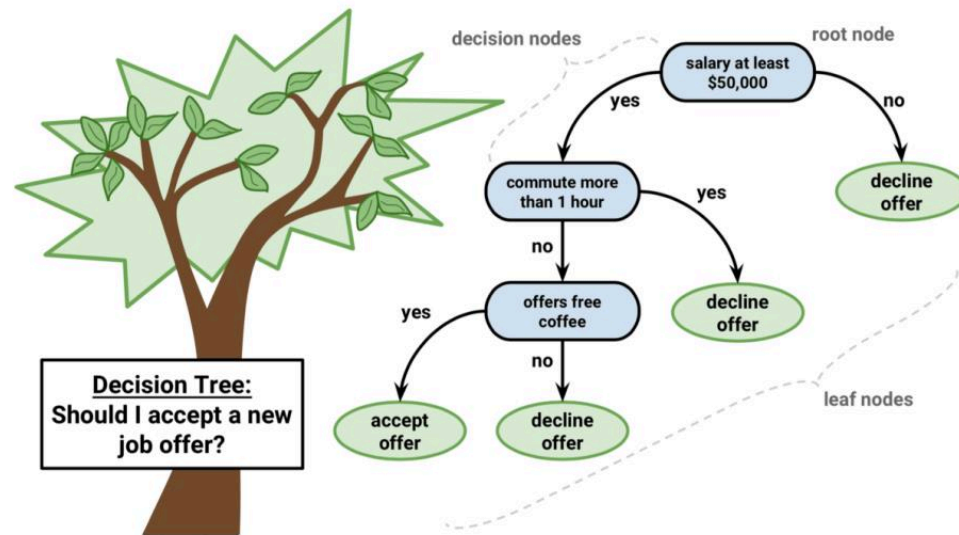
The data may not perfectly linearly separable.

We set C to find a balance between keeping the margin as large as possible and limiting the number of margin violations (instances that end up within the margin or on the wrong side).

Decision Trees

Decision Trees are a non-parametric supervised learning method used for both classification and regression.

- Pros: Interpretable (“white box” model), Handles both numerical and categorical data, No feature scaling required, Low data dependency
- Cons: Prone to overfitting, Can be unstable (small data variations can result in a completely different tree), Can struggle with certain concepts (e.g., XOR)



To make a prediction, an instance traverses the tree from the root node down to a leaf node.

Each internal node asks a true/false question about a single feature.

The prediction is the majority class (for classification) or the mean value (for regression) of the training samples in that leaf.

Decision Trees Example in scikit-learn

Scikit-learn provides two main classes for Decision Trees:

- **'DecisionTreeClassifier'** for classification tasks.
- **'DecisionTreeRegressor'** for regression tasks.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# max_depth is a key regularization hyperparameter
tree_clf = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_clf.fit(X_train, y_train)
y_pred = tree_clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
# Accuracy: 1.00
```

Decision Trees Example in scikit-learn

Scikit-learn provides two main classes for Decision Trees:

- 'DecisionTreeClassifier' for classification tasks.
- 'DecisionTreeRegressor' for regression tasks.

```
from sklearn.tree import DecisionTreeRegressor
import numpy as np

# Create some dummy regression data
X = np.array([[0, 0], [2, 2]])
y = np.array([0.5, 2.5])

# Train the regressor
tree_reg = DecisionTreeRegressor()
tree_reg.fit(X, y)

# Predict the value for a new point
new_point = [[1, 1]]
print(tree_reg.predict(new_point)) # [0.5]
```

The prediction for a new instance is the average target value of the training samples in the leaf node it falls into.

Visualizing a Decision Tree

One of the greatest strengths of Decision Trees is their interpretability. We can visualize the trained tree using Scikit-learn's 'plot_tree' function.

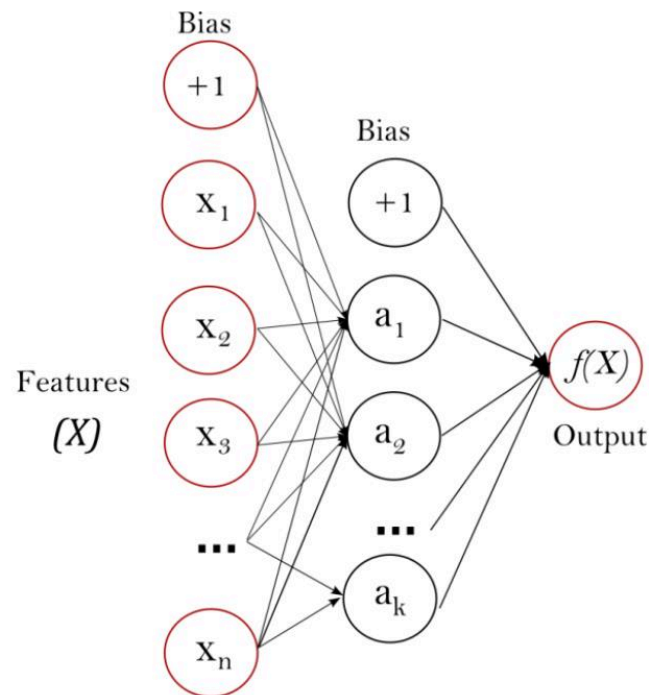
Code to Plot the Tree

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(12, 8))
plot_tree(
    tree_clf, # The trained model from the previous slide
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True
)
plt.title("Decision Tree for Iris Classification")
plt.show()
```

Multi-layer Perceptron (MLP)

A Multi-layer Perceptron (MLP) is a type of fully connected, feedforward artificial neural network. It is a versatile model capable of learning complex, non-linear functions.



MLP Example in scikit-learn

- Scikit-learn provides 'MLPClassifier' and 'MLPRegressor'.
- A critical preprocessing step for MLPs is feature scaling

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# It is highly recommended to use a pipeline for scaling
mlp_pipeline = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(100, 50), # Two hidden layers
        max_iter=1000,
        random_state=42
    )
)

mlp_pipeline.fit(X_train, y_train)
print(f"MLP Accuracy: {mlp_pipeline.score(X_test, y_test):.2f}")
```

Supervised Learning Models

- Linear Regression
- Logistic Regression
- Support Vector Machines (SVM)
- Decision Trees
- Multi-layer Perceptron (MLP)

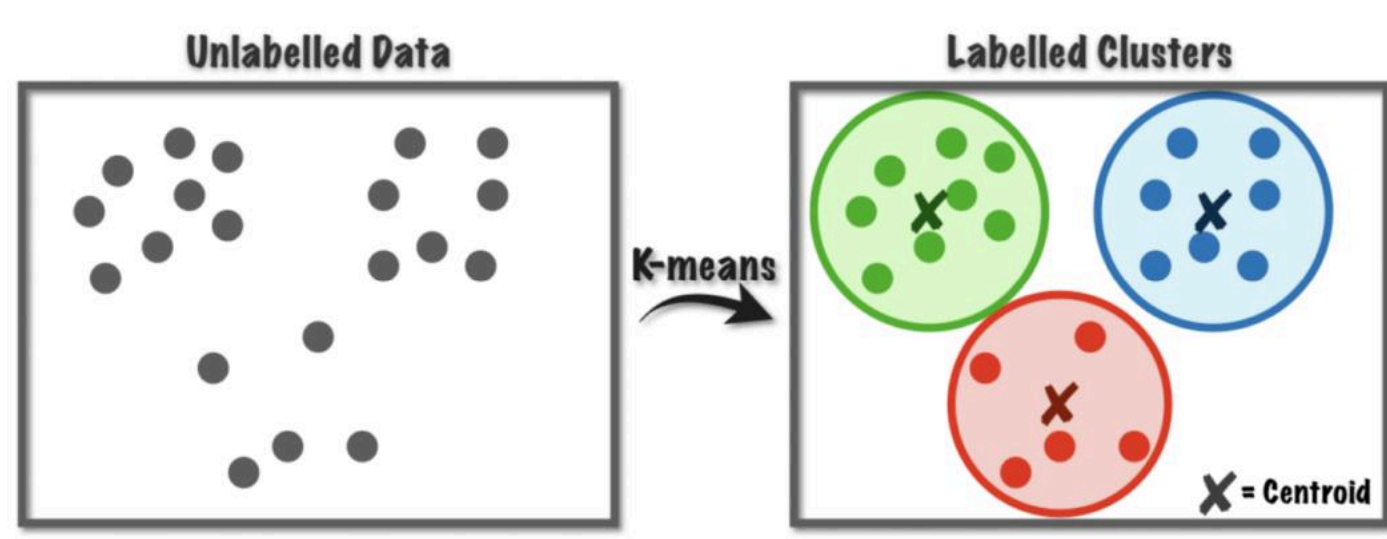
Unsupervised Learning Models (Clustering)

- K-Means
- Agglomerative Clustering
- DBSCAN
- Spectral Clustering
- Gaussian Mixtures

Unsupervised Learning: Clustering

Goal: To find hidden patterns or groupings in data without the need for pre-labeled outcomes.

- Scikit-learn provides a wide variety of clustering algorithms.
- The choice of algorithm depends heavily on the structure of your data and the goals of your analysis.



K-Means Clustering

K-Means partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster center or centroid).

It is general-purpose, fast, and scalable, works well for even-sized, spherical clusters.

```
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
# Load the data (we ignore the true labels for clustering)
iris = load_iris()
X = iris.data

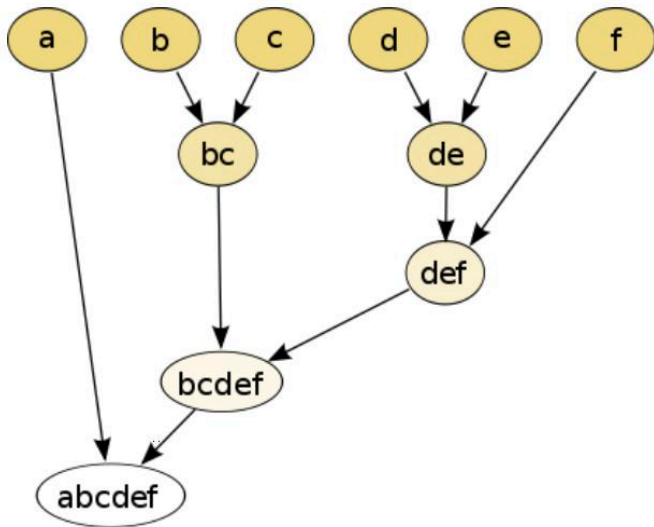
# We know there are 3 species, so we set n_clusters=3
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
# Train the model and get cluster assignments
kmeans.fit(X)
cluster_labels = kmeans.labels_
print("Cluster assignments for the first 10 samples:")
print(cluster_labels[:10])
# Predict the cluster for new data points
new_data = [[5.1, 3.5, 1.4, 0.2], [6.7, 3.0, 5.2, 2.3]]
predictions = kmeans.predict(new_data)
print("\nPredictions for new data:", predictions)
```

Agglomerative Clustering

A hierarchical clustering method that builds nested clusters by successively merging smaller clusters. Can work with various distance metrics

```
from sklearn.cluster import AgglomerativeClustering

clustering = AgglomerativeClustering(n_clusters=2).fit(X)
print("Agglomerative Clustering labels:", clustering.labels_)
```



■ Many More...

- **DBSCAN:** Can find non-flat geometry and unevenly sized clusters. Excellent at outlier removal.
- **Spectral Clustering:** Effective for non-flat geometry (e.g., nested circles). Based on graph distance.
- **Gaussian Mixtures:** A probabilistic model that assumes clusters come from different Gaussian distributions. Good for density estimation.



Model Evaluation & Selection

■ The Goal of Machine Learning

Question: What is our actual goal?

- Is it to get 100% accuracy on our training data?
- No. That's easy to “cheat” by memorizing the training set.
- Analogy: A student who memorizes 10 practice questions but doesn't grasp the underlying concepts will fail the real exam
- **Generalization** is the essence of learning — the ability to apply knowledge beyond the examples seen.

The True Goal of Generalization

Build a model that performs well on **new, unseen data** drawn from the **same distribution** as the training data.

How do we formalize "performing well"?

Risk Minimization:

- We have a loss function $\ell(f(x), y)$ that measures how bad a prediction $f(x)$ is compared to the true label y .
- We want to find a function f that minimizes the True Risk (or Expected Risk)
- This is the true average loss over all possible data points.

$$\mathcal{R}(f) = \mathbb{E}_{(x,y) \sim \mathcal{P}}[\ell(f(x), y)]$$

Empirical Risk Minimization (ERM)

- We do not know the true data distribution, all we have is a finite training set
- Thus, we minimize the Empirical Risk (the average loss on our training set) instead

$$\mathcal{R}_{\text{emp}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i), y_i)$$

■ The Core Tension: Underfitting vs. Overfitting

Minimizing empirical risk (training error) is a good proxy, but it leads to a fundamental tension:

Underfitting:

- Problem: The model is too simple to capture the underlying patterns in the data.
- Symptom: **High training error and high test error.**
- Analogy: The student who didn't study at all. Fails practice and real exams.

Overfitting:

- Problem: The model is too complex. It memorizes the training data, including its noise.
- Symptom: **Very low training error but high test error.**
- Analogy: The student who memorized the 10 practice questions. Aces the practice, fails the real exam.

Generalization is the "sweet spot" in between.

■ A Framework for Generalization: Bias & Variance

We can formally decompose the expected test error of a model:

Bias:

- The error from the model's "stubbornness" or incorrect assumptions.
- How far off is the average model (trained on all possible datasets) from the true function?
- **High Bias → Underfitting.** (e.g., using a linear model for a sine wave)

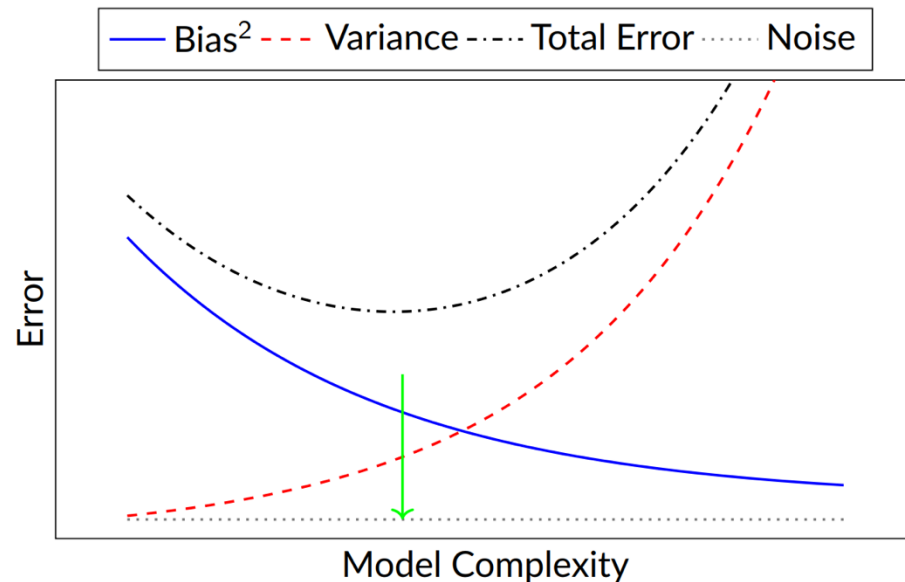
Variance:

- The error from the model's "sensitivity" to the specific training data it saw.
- How much do models (trained on different datasets) vary from each other?
- **High Variance → Overfitting.** (e.g., a high-degree polynomial that wiggles to catch every point)

The Bias-Variance Trade-off

There is a trade-off between bias and variance, controlled by **model complexity**.

- Low Complexity Model (e.g., Linear Regression)
 - Low Variance (all models look similar)
 - High Bias (likely wrong assumption)
- High Complexity Model (e.g., 15-degree Polynomial, deep NN)
 - High Variance (models are very sensitive to data)
 - Low Bias (can fit almost anything)



The Implementation of Generalization

Find the "sweet spot" of complexity that minimizes the **Total Error** ($\text{Bias}^2 + \text{Variance}$).

■ From Bias–Variance to Practice: Evaluation & Selection

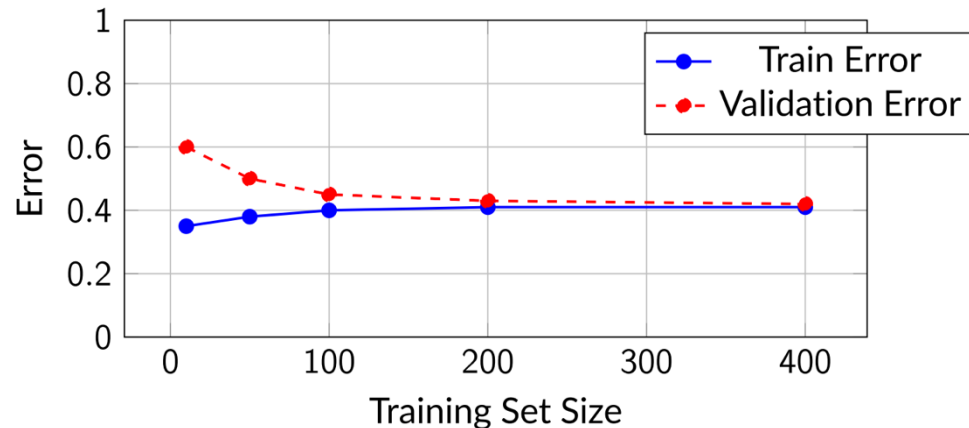
- Generalization is our end goal.
- Evaluation measures it; selection improves it.
- Without a sound protocol, we either underfit (high bias) or overfit (high variance), and reported numbers become misleading.

What do we want

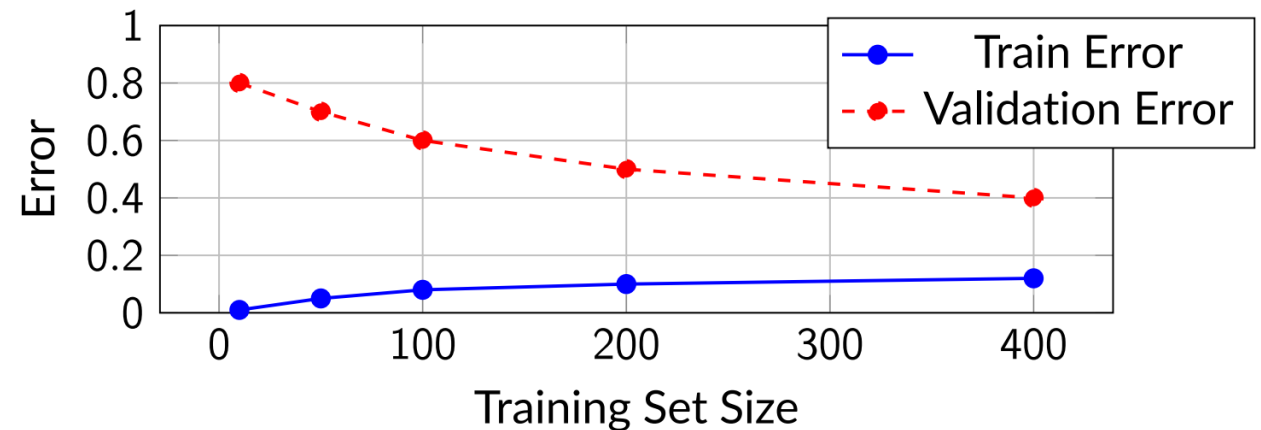
- Unbiased estimate of true risk (hold-out test).
- Low-variance selection signal (cross-validation).
- Alignment with bias–variance trade-off.

Design Principle

- Separate **what the algorithm learns (training)** from **what the practitioner decides (selection)**, and reserve a final **untouched test set** for reporting.
- Selection searches for the complexity/regularization that minimizes estimated test error.



Low complexity $\Rightarrow$ high bias: evaluation should reveal converged high train/val errors (diagnose underfitting).



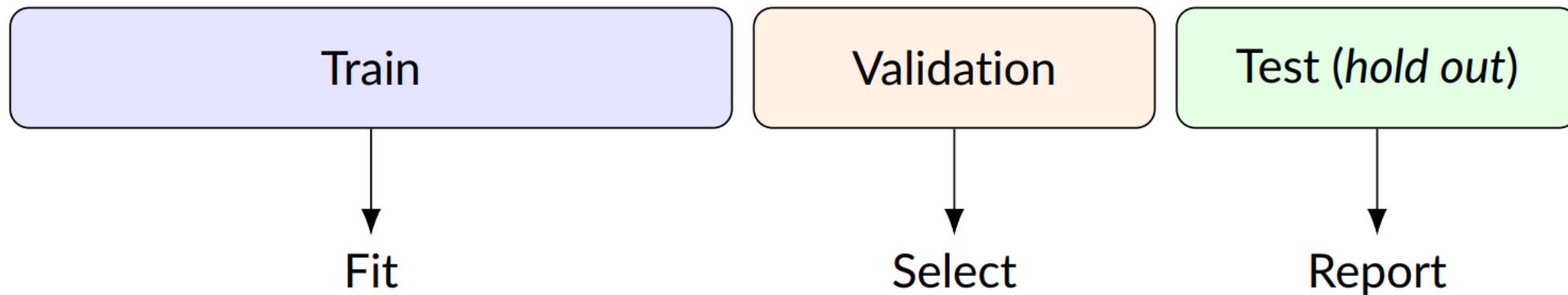
High complexity $\Rightarrow$ high variance: evaluation should reveal a gap between train and val errors (diagnose overfitting).

**Remedy: Get more data /
Add/increase regularization**

Clean Data-Splitting Protocol

- **Train:** Fit model parameters.
- **Validation:** Choose model type/hyper-parameters/early stopping.
- **Test (once):** Final, unbiased estimate.

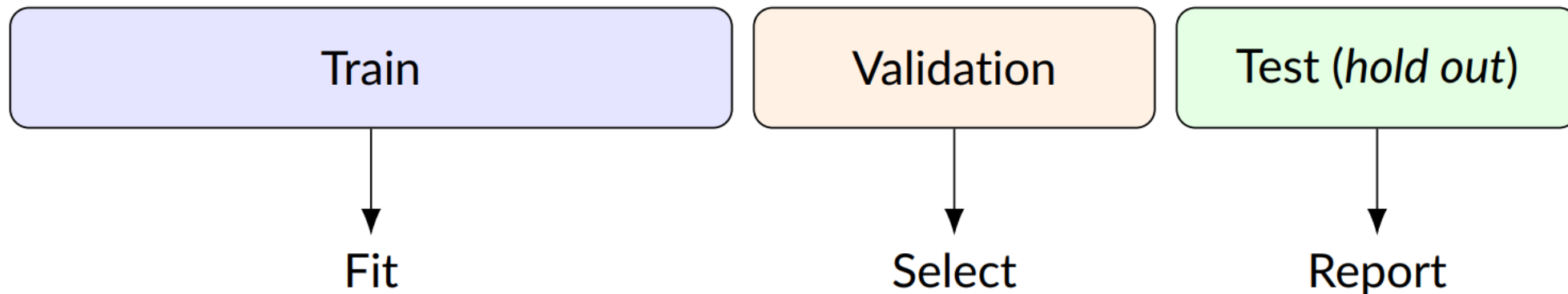
Example split: 60% Train / 20% Val / 20% Test (for large N).



Clean Data-Splitting Protocol

What can go wrong?

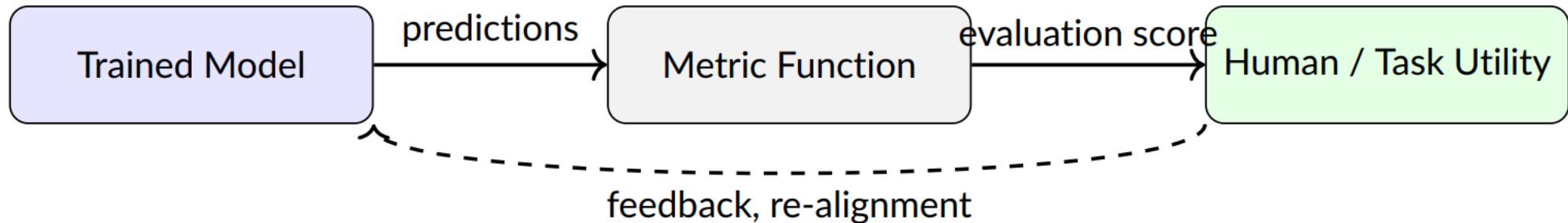
- **Unrepresentative Split:** Single random split yields brittle conclusions.
- **Leakage:** Using information from test in training or tuning. **Never use test data for training or selection. Use it once at the very end.**
- **Overfitting to Validation:** Re-using the same val split until it becomes a de-facto train set.



Model Evaluation: Why Metrics Matter

Metrics quantify how well the model aligns with what we truly care about in the downstream task.

- Provide an objective measurement of model behavior using quantitative criteria.
- Translate human goals (accuracy, safety, efficiency, fairness, satisfaction) into measurable signals.
- Serve as a feedback signal for improvement, selection, and deployment decisions.



Regression:

- Large-error risk $\Rightarrow$ MSE/RMSE
- Robustness $\Rightarrow$ MAE
- Logistic regression $\Rightarrow$ Log/Cross-Entropy Loss

Classification

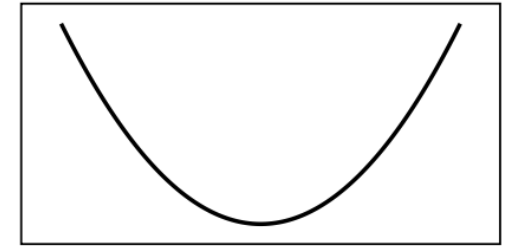
- Few positives $\Rightarrow$ PR-AP
- Balanced classes $\Rightarrow$ ROC-AUC
- Operating point needed $\Rightarrow$ Precision/Recall/F1

Regression: MSE & RMSE

Pros/Cons

- Pros: Smooth, differentiable; emphasizes impactful errors.
- Cons: Outlier-sensitive; scale-dependent.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}$$

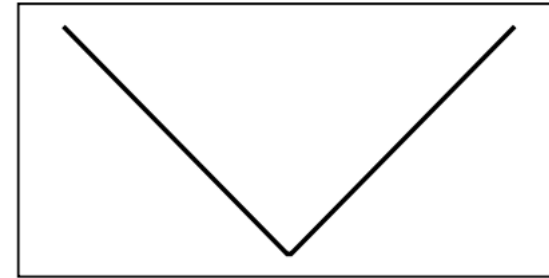


```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_true, y_pred, squared=True)
rmse = mean_squared_error(y_true, y_pred, squared=False)
```

Pros/Cons

- Pros: Robust to outliers; interpretable in original units; median-optimal.
- Cons: Non-smooth at 0; less sensitive to large errors..

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$



```
from sklearn.metrics import mean_absolute_error  
mae = mean_absolute_error(y_true, y_pred)
```

Regression: Log Loss (for Logistic Regression)

Log Loss (Binary Cross-Entropy) heavily penalizes confident wrong answers.

- If the true label $y = 1$, the cost is $-\log(p)$. The cost is huge if p is close to 0.
- If the true label $y = 0$, the cost is $-\log(1 - p)$. The cost is huge if p is close to 1.

Log Loss Cost Function $J(\theta)$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

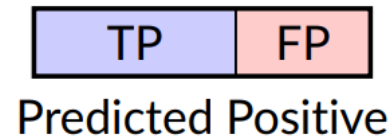
```
from sklearn.metrics import log_loss  
  
loss = log_loss(y_true, y_pred_prob)
```

Classification: Precision

Precision measures reliability of positive predictions, preferred when false alarms are costly.

- Pros: Direct proxy for alert reliability.
- Cons: Ignores missed positives (recall).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$



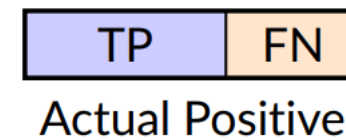
```
from sklearn.metrics import precision_score
prec = precision_score(y_true, y_pred_labels, average='
    binary')
```

Classification: Recall

Recall measures coverage of actual positives, preferred when missing positives is costly.

- Pros: Direct proxy for miss risk; important for safety/health applications.
- Cons: Ignores false positives (precision)

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$



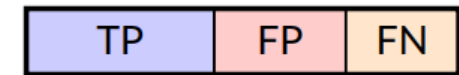
```
from sklearn.metrics import recall_score
rec = recall_score(y_true, y_pred_labels, average='
    binary')
```

Classification: F1

F1 balances precision and recall, preferred when false positives and false negatives are equally important.

- Pros: Single number for PR trade-off; robust to class imbalance.
- Cons: Ignores true negatives; hides threshold choice.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$



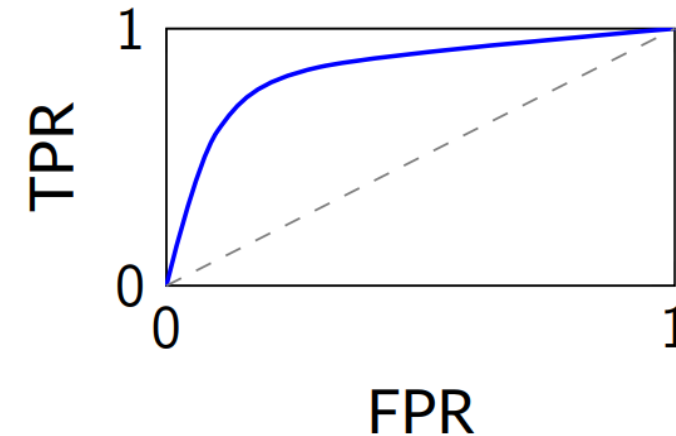
```
from sklearn.metrics import f1_score  
f1 = f1_score(y_true, y_pred_labels, average='binary')
```

Classification: ROC–AUC

ROC–AUC measures ranking quality, preferred when you want good ordering for later threshold tuning.

- Pros: Threshold-free; less sensitive to class imbalance.
- Cons: May hide poor performance on positive class under heavy imbalance.

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$



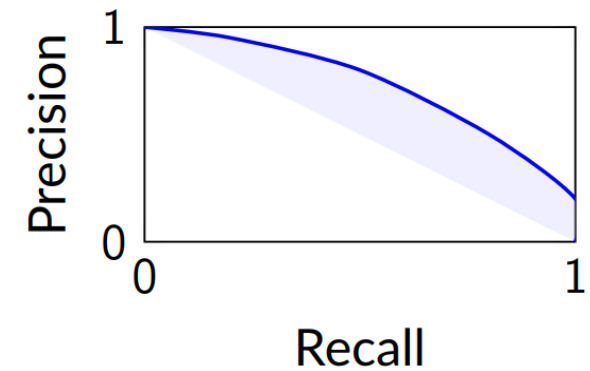
```
from sklearn.metrics import roc_auc_score, roc_curve
auc = roc_auc_score(y_true, y_scores)
fpr, tpr, thr = roc_curve(y_true, y_scores)
```

Classification: PR-AP

PR-AP summarizes precision-recall trade-off, preferred when positive class is rare and missing positives is costly.

- Pros: Reflects positive-class utility; useful for imbalanced tasks.
- Cons: Sensitive to prevalence; threshold still needed for deployment.

$$AP = \int_0^1 \text{Precision}(r) dr$$



```
from sklearn.metrics import average_precision_score,
    precision_recall_curve
ap = average_precision_score(y_true, y_scores)
prec, rec, thr = precision_recall_curve(y_true, y_scores
    )
```

Regression:

- Large-error risk $\Rightarrow$ MSE/RMSE
- Robustness $\Rightarrow$ MAE
- Logistic regression $\Rightarrow$ Log/Cross-Entropy Loss

Classification

- Few positives $\Rightarrow$ PR-AP 类别不平衡时
- Balanced classes $\Rightarrow$ ROC-AUC 类别平衡时
- Operating point needed $\Rightarrow$ Precision/Recall/F1 需要看threshold时

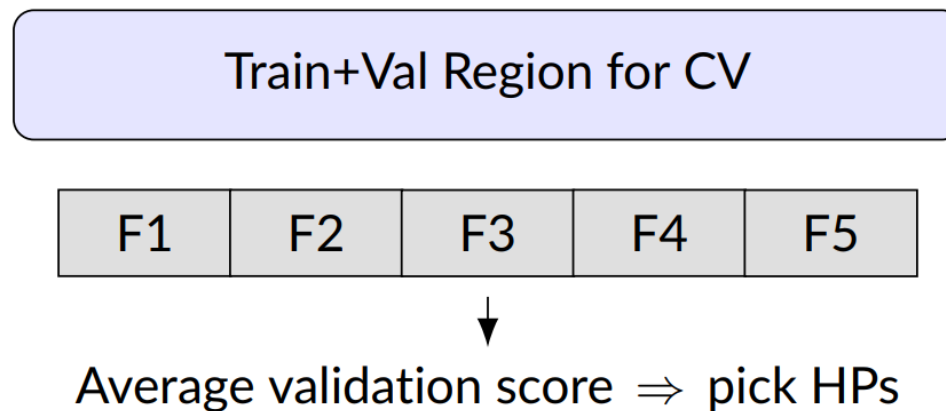
Always

- Report Confidence Interval (CI) or CV (Cross-Validation) $\text{mean} \pm \text{std}$
- Disclose threshold

Model Selection: Core Workflow

Goal: Pick model/hyper-parameters expected to generalize best.

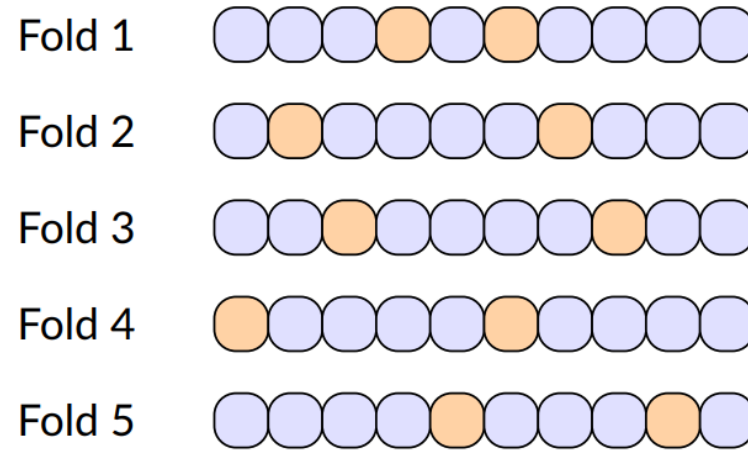
- Define search space (model types, hyper-parameters(HP) ranges)
- Use K-Fold cross-validation(CV) on Train+Val portion to estimate performance per setting
- Choose the setting with best average CV score
- Retrain the selected configuration on Train+Val
- Evaluate once on Test



- Stratified K-Fold
- Group K-Fold
- Time Series Split
- Nested Cross-Validation (for Unbiased Reporting)

Stratified K-Fold

Preserve class proportions per fold; reduces variance of selection signal.

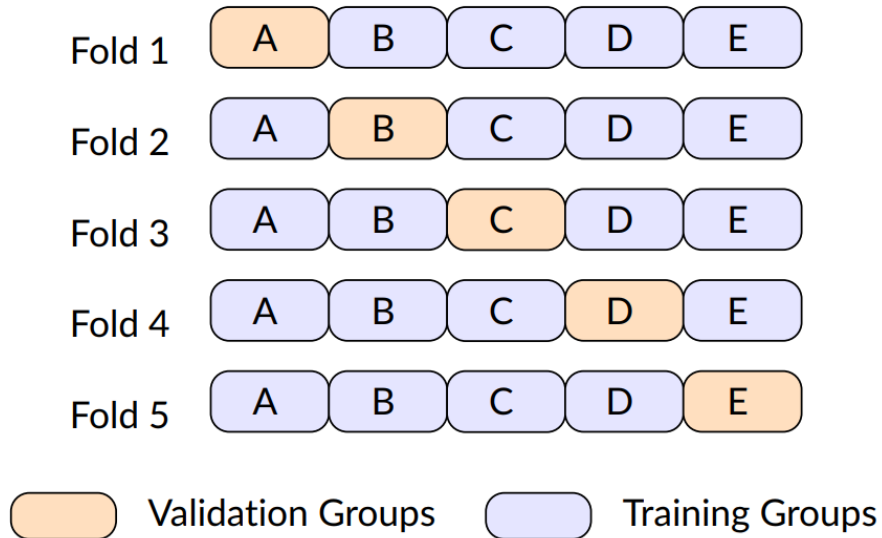


 Positive Class (minority)  Negative Class (majority)

```
from sklearn.model_selection import StratifiedKFold,
    cross_val_score
skf = StratifiedKFold(n_splits=5, shuffle=True,
    random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring='
    f1')
```

Group K-Fold

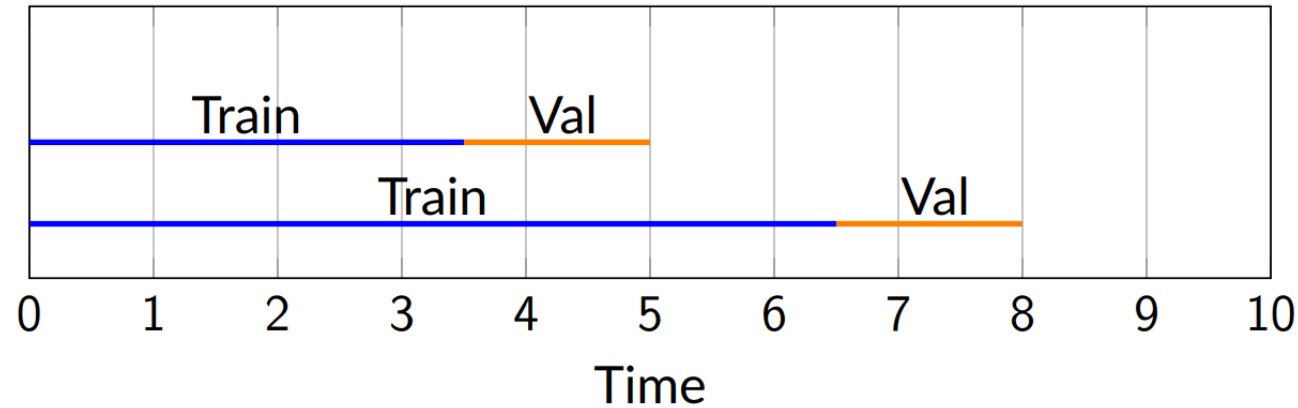
Keep all samples from the same entity (patient/user) in the same fold; avoids optimistic bias.



```
from sklearn.model_selection import GroupKFold,
    cross_val_score
# groups: array-like indicating entity id per sample
gkf = GroupKFold(n_splits=5)
scores = cross_val_score(model, X, y, groups=groups, cv=
    gkf, scoring='roc_auc')
```

Time Series Split

Train on the past, validate on the immediate future;
use rolling or expanding window; never shuffle



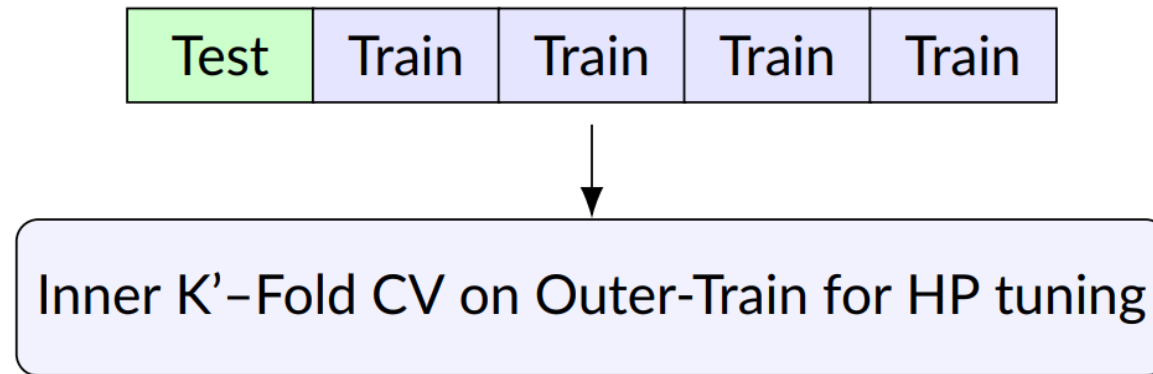
```
from sklearn.model_selection import TimeSeriesSplit,
    cross_val_score
tscv = TimeSeriesSplit(n_splits=5)
scores = cross_val_score(model, X, y,
                          cv=tscv, scoring='
                              neg_mean_squared_error')
rmse = (-scores.mean())**0.5
print(rmse)
```

Nested Cross-Validation (for Unbiased Reporting)

Problem: Using the same CV for both selection and reporting is optimistically biased.

Nested CV:

- Outer loop (K folds): evaluation folds.
- Inner loop (K'): selection & tuning within each outer-train split.
- Report mean $\pm$ std of outer test scores.





Hyperparameter Optimization

What Are Hyperparameters?

Configuration variables set outside empirical risk minimization.

Types:

- Continuous: learning rate, weight decay.
- Discrete: number of estimators, layers, neighbors.
- Categorical: optimizer choice, kernel type, activation.
- Conditional: HPs active only when another choice is made (e.g., if kernel=rbf then tune γ)

■ Why Hyperparameter(HP) Optimization?

Models expose hyperparameters (HPs) that control capacity, regularization, and training dynamics.

- **Accuracy Robustness:** HPs alter bias–variance; choosing them well is crucial to reach the generalization “sweet spot”.
- **Efficiency:** Training time, memory, and inference cost depend on HPs.
- **Reliability:** Stable, reproducible performance requires principled tuning protocols.
- **Fair Comparison:** Methods must be compared under comparable tuning budgets.

Consider model parameter θ , hyper-parameters λ

Clean separation

- Use Train to fit parameters θ for each λ .
- Use Validation (or inner CV) to select λ .
- Report once on Test (or outer CV) after selection.

When data are scarce

- **K-Fold CV** in the outer loop for selection signal with lower variance.
- **Nested CV** to obtain an unbiased generalization estimate after hyperparameter optimization.

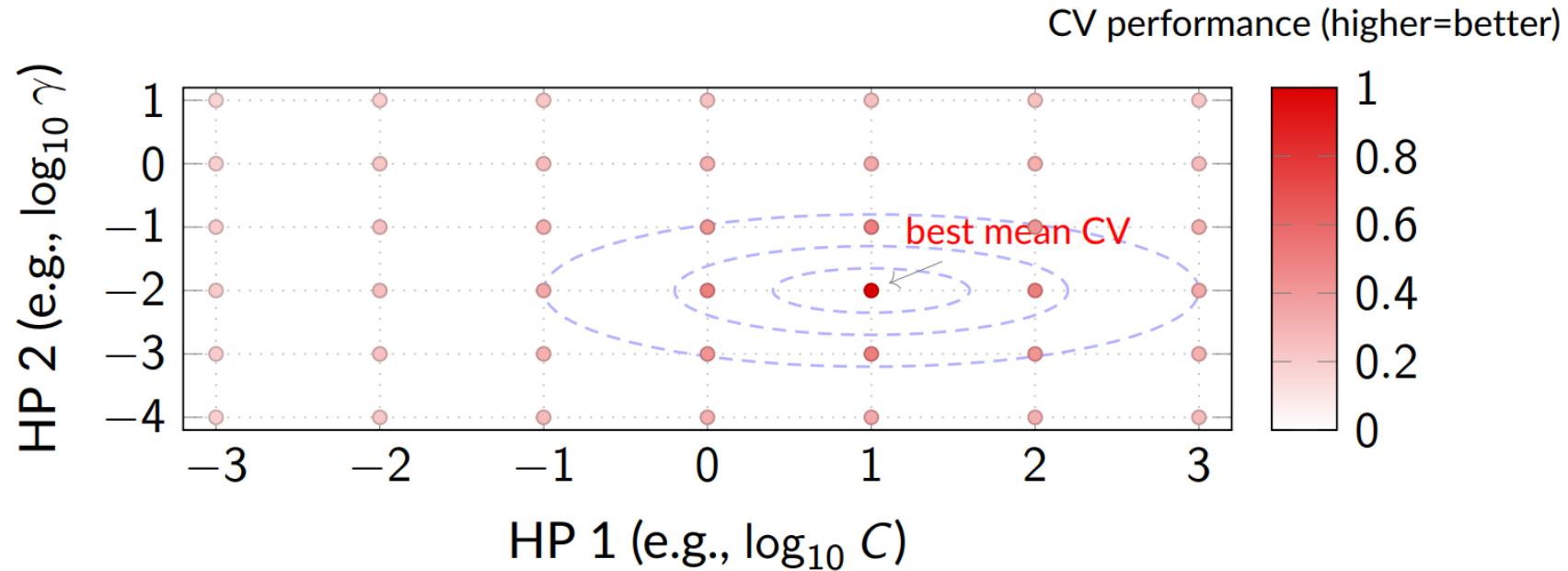
Reporting Norm: Disclose search space, algorithm, budget, CV protocol, seed(s), and the final test estimate with Confidence Interval (CI) or $\text{mean} \pm \text{std}$

- Grid Search
- Random Search
- Successive Halving

Grid Search

Idea: Evaluate a Cartesian grid over each HP dimension.

- Strength: deterministic, easy to parallelize, reproducible; good for very low dimension and when interactions are known.
- Weakness: Curse of dimensionality; wastes trials on unimportant dimensions; sensitive to grid choice.



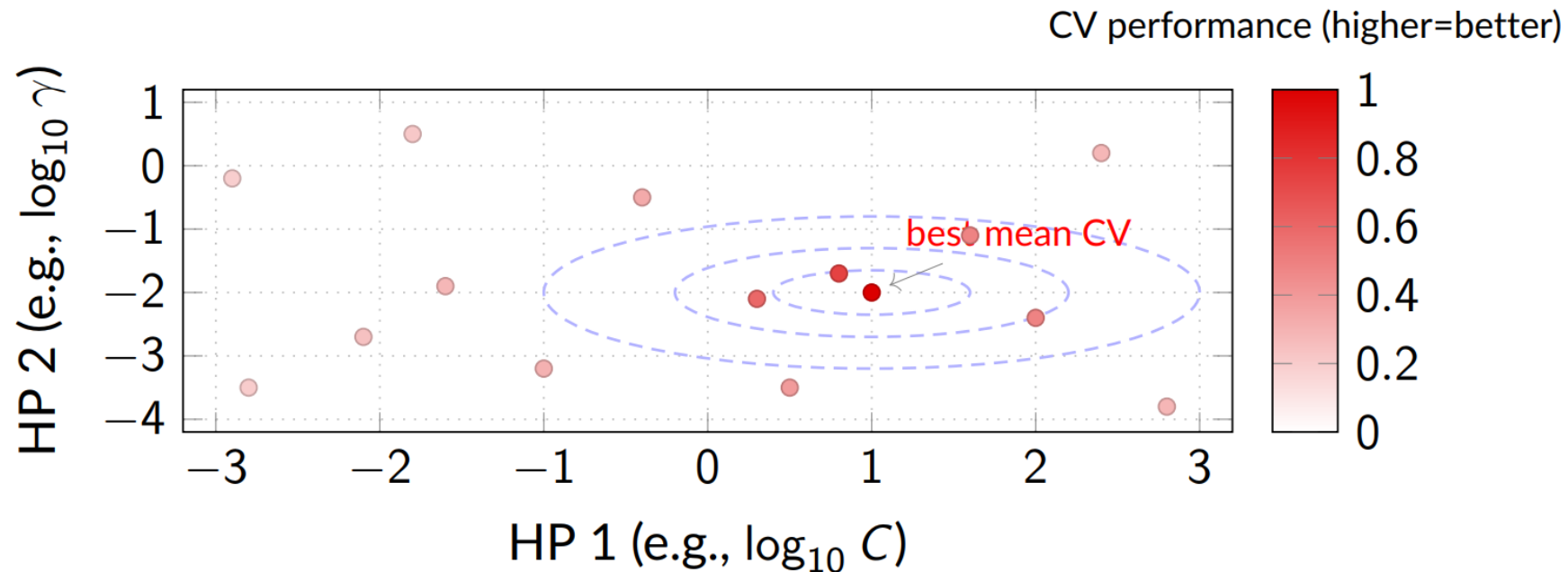
Grid Search Example in scikit-learn

```
from sklearn import svm, datasets
from sklearn.model_selection import GridSearchCV,
    train_test_split
# Load example data
X, y = datasets.load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y
    , test_size=0.3, random_state=42)
# Define model and hyperparameter grid
svc = svm.SVC()
param_grid = {'C': [0.1, 1, 10, 100], 'gamma': [1, 0.1,
    0.01, 0.001], 'kernel': ['rbf']}
# Perform grid search with 5-fold cross-validation
grid = GridSearchCV(svc, param_grid, refit=True, cv=5,
    verbose=0)
grid.fit(X_train, y_train)
print("Best parameters:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)
print("Test accuracy:", grid.score(X_test, y_test))
```

Random Search

Idea: Instead of evaluating every grid point, sample random configurations from the hyperparameter space.

- Strength: More efficient in high-dimensional spaces; good for finding near-optimal regions quickly.
- Weakness: Results may vary due to randomness; may miss small but important regions.



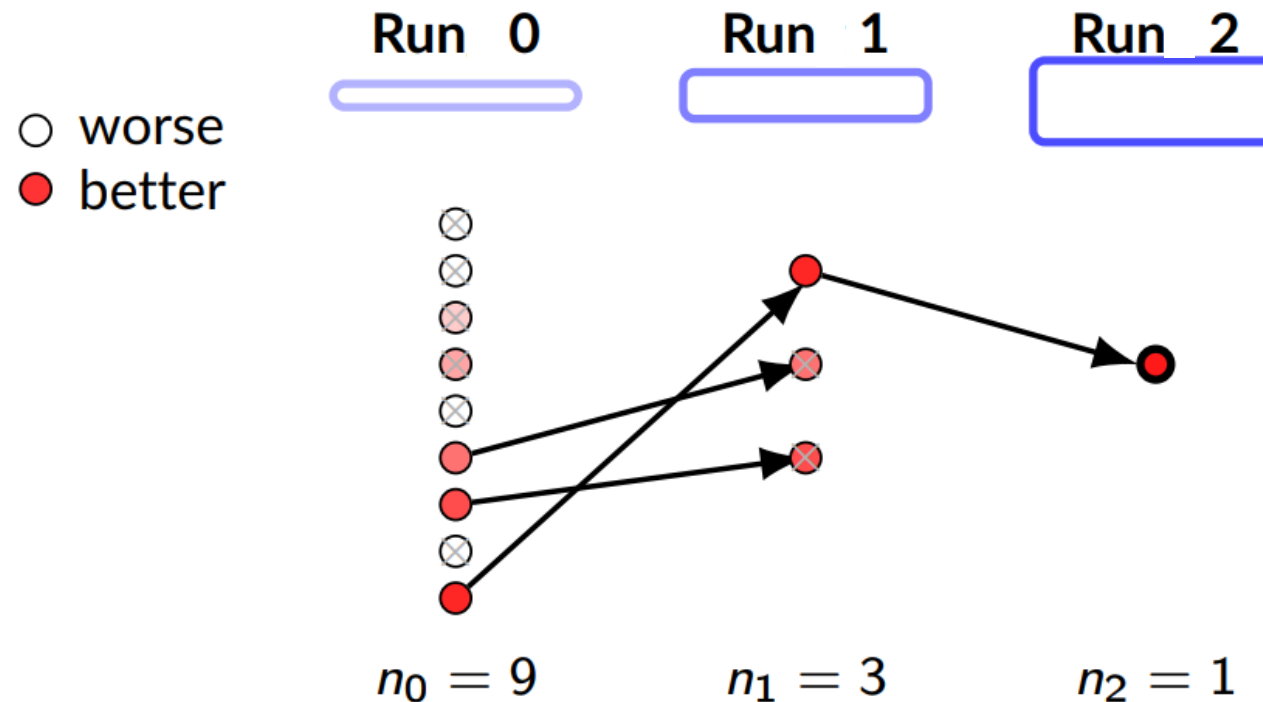
Random Search Example in scikit-learn

```
from sklearn import svm, datasets
from sklearn.model_selection import RandomizedSearchCV,
    train_test_split
from scipy.stats import loguniform
# Load example data
X, y = datasets.load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3, random_state=42, stratify=y)
# Define model and hyperparameter distributions
svc = svm.SVC()
param_distributions = {'C': loguniform(1e-2, 1e2), 'gamma': loguniform
    (1e-4, 1e0), 'kernel': ['rbf']}
# Perform random search with 5-fold cross-validation
rand = RandomizedSearchCV(estimator=svc, param_distributions=
    param_distributions, n_iter=20, cv=5, random_state=42, n_jobs=-1,
    verbose=0)
rand.fit(X_train, y_train)
print("Best parameters:", rand.best_params_)
print("Best CV accuracy:", rand.best_score_)
print("Test accuracy:", rand.score(X_test, y_test))
```

Successive Halving

Idea: Start many configs with a small budget; keep the top $1/\eta$ and increase the budget per surviving config at each run.

When Useful: Training curves are predictive early (learning-curve extrapolation); large search spaces; limited total budget.



Successive Halving Example in scikit-learn

```
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import HalvingGridSearchCV,
    HalvingRandomSearchCV, train_test_split
from sklearn import svm, datasets
from scipy.stats import loguniform
X, y = datasets.load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
    random_state=42, stratify=y)
svc = svm.SVC()
param_dist = {'C': loguniform(1e-2, 1e2), 'gamma': loguniform(1e-4, 1
    e0), 'kernel': ['rbf']}
shr = HalvingRandomSearchCV(estimator=svc, param_distributions=
    param_dist, factor=3, resource='n_samples', max_resources='auto',
    min_resources=50, random_state=42, n_candidates='exhaust', scoring
    ='accuracy')
shr.fit(Xtr, ytr)
print("SH best params:", shr.best_params_, "CV acc:", shr.best_score_)
print("Test acc:", shr.score(Xte, yte))
```

Well-posed search spaces (ranges, scales, conditions) matter as much as the search algorithm.

- **Right scale:** tune on log-scale for multiplicative HPs (e.g., learning rate $\in [10^{-5}, 10^{-1}]$ sampled uniformly in $\log_{10}$).
- **Ranges from priors:** literature, sanity checks, and pilot runs inform bounds (e.g., set dropout $\in [0.0, 0.5]$ because prior work rarely exceeds 0.5).
- **Conditional structure:** encode dependencies to avoid invalid configs (e.g., if kernel=rbf then tune γ ; if kernel=linear then skip γ).
- **Normalization:** map to $[0, 1]^d$ to help model-based methods (e.g., optimize $u \in [0, 1]$ and set $lr = 10^{-5+4u}$, $depth = 2 + 18u$).
- **Compactness vs. coverage:** too wide wastes budget; too narrow risks misspecification (e.g., start with $lr \in [10^{-4}, 10^{-1}]$; widen only if the best lies at the boundary)

In-Class Practice (Submit to Canvas)

Objective: Tune an SVC with Successive Halving Random Search.

Fixed setup

- Dataset: `sklearn.datasets.breast_cancer` for binary classification.
- Split: 70% Train / 30% Test, stratified, `random_state=42`.
- Estimator: `Pipeline(StandardScaler(), SVC(kernel='rbf', probability=True))`.
- Optimizer: `sklearn.model_selection.HalvingRandomSearchCV` with `cv=StratifiedKFold(5, shuffle=True, random_state=42)`, `scoring='roc_auc'`, `n_jobs=-1`, `random_state=42`.
- Search space: $C \in [10^{-2}, 10^2]$, $\gamma \in [10^{-4}, 10^0]$, both sampled log-uniformly.
- Successive halving setting: Use `factor=3`, `resource='n_samples'`, and let the search allocate more training samples to promising configurations.

What to implement

1. Fit `HalvingRandomSearchCV` on the Train split.
2. Refit the pipeline on the full Train split using the best hyperparameters.
3. Evaluate once on the Test split.
4. Print: best hyperparameters; number of candidates evaluated at each halving iteration; best cross-validation ROC-AUC, reported as $\text{mean} \pm \text{std}$ over 5 folds for the selected configuration at its final halving iteration; Test ROC-AUC

ML Basics with Scikit-Learn

- Load Dataset, Split Data
- Preprocess Data: Scale, Normalization, Encoding, Discretization, Binarization, Imputation, Generating Polynomial Features, Dimensionality Reduction

Models in Scikit-Learn

- Supervised Learning Models: Linear/Logistic Regression, SVM, Decision Trees, MLPs
- Unsupervised Learning Models: K-Means, Agglomerative Clustering, ...

Model Evaluation & Selection

- Generalization: The Bias-Variance Trade-off
- Data-Splitting Protocol
- Evaluation Metric: MSE,, RMSE, MAE, log loss, ROC-AUC, PR-AP, Precision/Recall/F1
- Cross-Validation: Stratified K-Fold, Group K-Fold, Time Series Split, Nested Cross-Validation

Hyperparameter Optimization

- Search Algorithm: Grid Search, Random Search, Successive Halving
- Search Space Engineering

Python Programming with Artificial Intelligence

Thank you for listening!

